

DBMS – BUSINESS INTELLIGENCE AND AI DATA PREPARATION

Project Title: EcoCart – Sustainable Products E-Commerce Order Management System

Skill Task: DBMS

Week: 14

Objectives

1. Create analytical reports using SQL.
2. Apply window functions (RANK, ROW_NUMBER).
3. Generate customer segmentation data.
4. Prepare datasets for AI models.
5. Perform feature extraction using SQL.

1. Create Analytical Reports Using SQL

To generate meaningful business reports from the EcoCart database using aggregate functions, joins, grouping, and sorting.

```
SQL> SELECT c.customer_id,  
2      c.customer_name,  
3      COUNT(o.order_id) AS total_orders,  
4      SUM(o.total_amount) AS total_spent,  
5      AVG(o.total_amount) AS average_order_value  
6 FROM customer c  
7 LEFT JOIN orders o  
8     ON c.customer_id = o.customer_id  
9 GROUP BY c.customer_id,  
10        c.customer_name  
11 ORDER BY total_spent DESC;
```

```
CUSTOMER_ID  
-----  
CUSTOMER_NAME  
-----  
TOTAL_ORDERS TOTAL_SPENT AVERAGE_ORDER_VALUE  
-----  
101  
Anitha Kumar  
4      17700      4425  
103  
Priya  
2      12200      6100
```

```
CUSTOMER_ID  
-----  
CUSTOMER_NAME  
-----  
TOTAL_ORDERS TOTAL_SPENT AVERAGE_ORDER_VALUE  
-----  
102  
Rahul  
1      1800      1800
```

2. Apply Window Functions

Window functions perform calculations across related rows without combining them into a single row.

2.1 RANK()

To rank customers according to their total spending.

```
SQL> SELECT customer_id,  
2         customer_name,  
3         total_spent,  
4         RANK() OVER (  
5             ORDER BY total_spent DESC  
6         ) AS spending_rank  
7 FROM  
8 (  
9     SELECT c.customer_id,  
10         c.customer_name,  
11         NVL(SUM(o.total_amount), 0) AS total_spent  
12 FROM customer c  
13 LEFT JOIN orders o  
14     ON c.customer_id = o.customer_id  
15 GROUP BY c.customer_id,  
16         c.customer_name  
17 );
```

CUSTOMER_ID

CUSTOMER_NAME

TOTAL_SPENT SPENDING_RANK

101	
Anitha Kumar	
17700	1
103	
Priya	
12200	2

CUSTOMER_ID

CUSTOMER_NAME

2.2 ROW_NUMBER()

ROW_NUMBER() assigns a unique sequential number to each row.

```
SQL> SELECT customer_id,
2         customer_name,
3         total_spent,
4         ROW_NUMBER() OVER (
5             ORDER BY total_spent DESC
6         ) AS row_number
7 FROM
8 (
9     SELECT c.customer_id,
10            c.customer_name,
11            NVL(SUM(o.total_amount), 0) AS total_spent
12     FROM customer c
13     LEFT JOIN orders o
14         ON c.customer_id = o.customer_id
15     GROUP BY c.customer_id,
16             c.customer_name
17 );
```

CUSTOMER_ID

CUSTOMER_NAME

TOTAL_SPENT ROW_NUMBER

101		
Anitha Kumar		
17700	1	
103		
Priya		
12200	2	

CUSTOMER_ID

CUSTOMER_NAME

3. Generate Customer Segmentation Data

To divide customers into meaningful groups based on their purchasing behavior.

We can use **total spending** as the segmentation feature.

```
SQL> SELECT customer_id,
2         customer_name,
3         total_spent,
4         CASE
5             WHEN total_spent >= 10000
6                 THEN 'High Value'
7             WHEN total_spent >= 5000
8                 THEN 'Medium Value'
9             ELSE 'Low Value'
10        END AS customer_segment
11 FROM
12 (
13     SELECT c.customer_id,
14            c.customer_name,
15            NVL(SUM(o.total_amount), 0) AS total_spent
16     FROM customer c
17     LEFT JOIN orders o
18         ON c.customer_id = o.customer_id
19     GROUP BY c.customer_id,
20            c.customer_name
21 );
```

CUSTOMER_ID

CUSTOMER_NAME

TOTAL_SPENT CUSTOMER_SEG

103	
Priya	12200 High Value
104	
Karthik S	1500 Low Value

4. Prepare Dataset for AI Models

To transform raw database information into a structured dataset that can be used as input for machine learning or AI applications.

For example, an AI model could predict whether a customer is likely to be a **High Value**, **Medium Value**, or **Low Value** customer.

```
SQL> SELECT c.customer_id,  
2      COUNT(o.order_id) AS total_orders,  
3      NVL(SUM(o.total_amount), 0) AS total_spent,  
4      NVL(AVG(o.total_amount), 0) AS average_order_value,  
5      CASE  
6          WHEN NVL(SUM(o.total_amount), 0) >= 10000  
7              THEN 2  
8          WHEN NVL(SUM(o.total_amount), 0) >= 5000  
9              THEN 1  
10         ELSE 0  
11     END AS customer_segment  
12 FROM customer c  
13 LEFT JOIN orders o  
14     ON c.customer_id = o.customer_id  
15 GROUP BY c.customer_id;
```

CUSTOMER_ID	TOTAL_ORDERS	TOTAL_SPENT	AVERAGE_ORDER_VALUE	CUSTOMER_SEGMENT
101	4	17700	4425	2
102	1	1800	1800	0
103	2	12200	6100	2
104	1	1500	1500	0

5. Perform Feature Extraction Using SQL

To extract useful features from raw customer and order data.

A **feature** is a measurable value that represents an important characteristic of a customer, product, or transaction.

For EcoCart, useful customer features include:

- Number of orders
- Total spending
- Average order value
- Maximum order value
- Minimum order value

```
SQL> SELECT c.customer_id,
2         c.customer_name,
3         COUNT(o.order_id) AS total_orders,
4         NVL(SUM(o.total_amount), 0) AS total_spent,
5         NVL(AVG(o.total_amount), 0) AS average_order_value,
6         NVL(MAX(o.total_amount), 0) AS maximum_order_value,
7         NVL(MIN(o.total_amount), 0) AS minimum_order_value
8 FROM customer c
9 LEFT JOIN orders o
10      ON c.customer_id = o.customer_id
11 GROUP BY c.customer_id,
12          c.customer_name;
```

CUSTOMER_ID

CUSTOMER_NAME

TOTAL_ORDERS TOTAL_SPENT AVERAGE_ORDER_VALUE MAXIMUM_ORDER_VALUE

MINIMUM_ORDER_VALUE

	103			
Priya	2	12200	6100	9000
		3200		

CUSTOMER_ID

CUSTOMER_NAME

TOTAL_ORDERS TOTAL_SPENT AVERAGE_ORDER_VALUE MAXIMUM_ORDER_VALUE

MINIMUM_ORDER_VALUE